N-gram Hints in Language Model Training

Vuk Rosić

Abstract

An n-gram hint tells a language model which token most often followed the last n tokens in the training data. Hints recently cut the NanoGPT Speedrun record from 39.9 s to 6.7 s, but training uses only hints from tokens trained in earlier steps, because a large corpus from the start makes the model rely on hints too much. In small Muon-trained models we show that this failure comes from a token's own occurrence in the counts: keeping it gives a validation loss of 3.53, while removing it and dropping half of the hints gives 1.80, against 1.89 for hints from earlier steps. On web text the token's whole page must be removed. Hints give perfect recall of stored facts, but a model that always gets hints memorizes almost nothing itself, and hints at all three places stop it from learning comparison. In a small test this block comes from wrong hints read before the answer, not from the hint at the answer. When one model trained on web text and reasoning together, a switch trained to predict whether each hint is right raised comparison from chance to at least .905 at three model sizes and lowered web text loss below plain hints by 0.04 to 0.09 at the longest training we ran. All models are small and each setting ran once. The code is public [23].

1 Introduction

Language models learn slowly from text they have seen only a few times, and web text is full of passages that repeat only occasionally. Brunborg [1] uses this: during training, idle CPUs build a corpus that records which tokens follow each n-gram. The model receives the most common next token as a hint and learns how much to trust it. The record went from 39.9 s to 6.7 s [2]. The blog post reports two things that matter for this paper. A large hint corpus from the start makes the backbone rely on the hints too much, so training uses a small corpus and validation a large one. And each doubling of the corpus cuts the loss more than the last [1].

The record's code computes training hints in a strictly causal way: a step's hints count only tokens trained in earlier steps [3]. That has a cost. At validation the model meets hints from a far larger corpus than any it trained with, so it must trust counts it has never seen.

We ask whether a language model trained with Muon can use hints from the whole corpus from the first step, without the failure the blog post describes, and what hints do to memorizing and reasoning. We make four contributions.

1. We show that a token's own occurrence in the counts is enough to break training with a full corpus.
2. We show that removing it, plus random hint dropout, beats causal hints at three model sizes.
3. We measure how the gain depends on the ratio of corpus size to tokens trained.
4. We test what hints help, facts or reasoning, find what makes hints block reasoning, and test fixes on web text.

2 Background

Muon. Muon [4] is an optimizer for the matrices of a network. It keeps momentum, replaces the update with its nearest orthogonal matrix using Newton-Schulz steps, and moves the weights by a fixed step [5]. Wang et al. [6] find that Muon learns rare classes better than Adam, the same rare-data problem hints address. Embeddings stay on Adam.

Memory by lookup. Engram [7] adds hashed n-gram embedding tables whose vectors are learned, and Tensorizing Engram [8] and Lngram v2 [9] extend this line. Hints differ: nothing is learned per n-gram, and the corpus stores only the next tokens. RETRO [10] retrieves whole text chunks from a very large database.

Leakage. A count feature that includes the row's own label leaks the label, as in target encoding [11]. CatBoost [12] fixes this with ordered statistics, which use only earlier rows, the same idea as causal hints. Leave-one-out statistics are the other common fix, and RETRO filters out neighbours from the same document [10]. Our searches found no paper comparing causal and leave-one-out hints for n-gram hints, and none using hint dropout, which follows dropout [13].

3 Method

Model. We use the Muon lab model [14]: a character-level transformer on Tiny Shakespeare with RoPE, QK-norm, RMSNorm, a SwiGLU MLP, no biases and tied embeddings, with 4 layers and 4 heads. The first 90% of the text is the training split. Muon trains the hidden matrices with momentum 0.95, and Adam trains the embeddings and norm weights at 0.003. Training has 20 warmup steps and a linear cooldown over the last 30%.

Hints. We follow the blog post, with lengths 6, 12 and 24 characters in place of 6, 10 and 20 tokens. For each length the hint is the most common next character. Its count falls into one of 7 buckets and its purity (the share of the n-gram's occurrences followed by that character) into one of 6, giving 126 kinds. Each kind has a learned amount σ and a learned vector β. The hint vector is

$$H_t = \sum_n \left(\sigma_{k(n,t)}\, E[\mathrm{hint}_n(t)] + \beta_{k(n,t)}\right), \tag{1}$$

where E is the embedding table. We add H_t after the embedding, after layer 2 and before the output head, each with its own learned scale. The hint parameters use Adam at 0.03.

Training hint policies. Validation always uses the whole training split as its corpus. Training uses one of these:

- **None:** no hints.
- **Own token kept:** the whole training split, with each token's own occurrence still in the counts.
- **Earlier steps:** only tokens trained in earlier steps, as in the record.
- **Leave-one-out:** the whole training split with each token's own occurrence removed. If removing it leaves no count, there is no hint, exactly as for an unseen n-gram at validation.

- **Leave-one-out with dropout:** the same, and each hint is dropped with probability 0.5.

Training streams shuffled, non-overlapping chunks of 65 characters, so a token appears at most once per pass. The default batch is 8 and the default length is 200 steps, or 102K trained tokens. The corpus is 1.0M characters, 10 times the tokens trained. The record's ratio is about 520 times [1], which a 1 MB text cannot reach.

Measurement. Each setting was run once at Muon learning rates 0.01, 0.02 and 0.04, and we report the best, on 96 fixed validation chunks.

4 Results

4.1 Keeping the token's own occurrence breaks training

Table 1 shows the results at width 64. With the token's own occurrence in the counts, every training n-gram has a match, and a long n-gram usually has only that one occurrence, so the hint is the answer. The model learns to trust hints it will not get at validation, and its loss with hints is 3.53 against 2.16 with no hints at all. Removing that one occurrence brings the loss to 1.84.

Training hints	Validation, hints on	Validation, hints off
None	–	2.165
Own token kept	3.525	4.735
Earlier steps	1.890	2.307
Earlier steps, half dropped	1.860	2.204
Leave-one-out	1.843	2.508
Leave-one-out, quarter dropped	1.800	2.349
Leave-one-out, half dropped	**1.805**	**2.256**
Leave-one-out, three quarters dropped	1.807	2.206

Table 1: Width 64, 200 steps, batch 8. Best of three learning rates by loss with hints.

4.2 Hint dropout removes most of the reliance on hints

Leave-one-out alone makes the backbone worse when hints are off: 2.508 against 2.165 with no hints, which matches the blog post's report of relying on hints too much. Dropping half of the hints brings this to 2.256 and lowers the loss with hints from 1.843 to 1.805. Dropout also helps earlier-steps hints (1.890 to 1.860), but they stay behind.

4.3 The gain stays the same as the model grows

Table 2 shows a gap between 0.070 and 0.085 across a 15-fold range of parameters.

4.4 The gain depends on corpus size divided by tokens trained

Table 3 changes the tokens trained at width 32 with the corpus fixed. The gap is 0.370 at a ratio of 40, 0.081 at 10 and 0 at 2.5, because hints from earlier steps approach the whole corpus as training goes on. At 3200 steps a second pass starts, so hints from earlier steps contain each token's own earlier occurrence and the failure above returns: loss rises to 2.533, while leave-one-out reaches 1.603. Causal hints are safe only within one pass.

Width	Parameters	No hints	Earlier steps	Leave-one-out, half dropped	Gap
32	56K	2.317	1.938	1.858	0.081
64	209K	2.165	1.890	1.805	0.085
128	812K	2.085	1.840	1.769	0.070

Table 2: Validation loss with hints on, 200 steps, batch 8, corpus 10 times the tokens trained.

Steps	Batch	Ratio	No hints	Earlier steps	Leave-one-out, half dropped	Gap
200	2	40	2.534	2.386	2.016	0.370
200	8	10	2.317	1.938	1.858	0.081
800	8	2.5	1.956	1.676	1.676	0.000
3200	8	0.6	1.755	2.533	1.603	0.930

Table 3: Width 32, hints on. Ratio is corpus size divided by tokens trained. The last row is past one pass over the data.

4.5 Each doubling of the corpus

With random 256-character pieces of the corpus, the gains per doubling are 0.040, 0.052, 0.050, 0.060, 0.099, 0.062 and 0.061 for leave-one-out with dropout, and 0.053, 0.055, 0.052, 0.060, 0.073, 0.028 and 0.018 for earlier steps (Figure 1). Neither curve shows gains that rise with each doubling, so we did not reproduce the blog post's result on Shakespeare. In the earlier-steps model the loss on hinted characters stops falling after 251K characters, about 2.5 times its tokens trained, because it never learned how much to trust larger counts. In the leave-one-out model it keeps falling to 1.66.

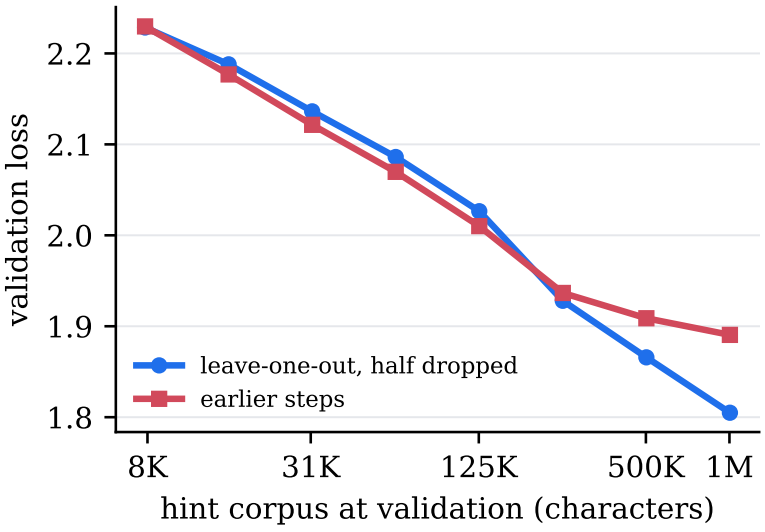


Figure 1: One final model of each kind evaluated with random subsets of the corpus. Width 64, 200 steps.

5 Web Text

We repeated the test on FineWeb [1], using GPT-2 tokens from one 100M-token training shard as the corpus and text from the separate validation shard, with corpus sizes from 0.78M to 100M tokens. The model has width 40, 4 layers, about 1.4M parameters, batch 8 and 90 steps (46K trained tokens) unless stated. Hints use lengths 6, 10 and 20 tokens unless stated; the longest set adds 2, 3, 4 and 5.

5.1 The hints improve with each doubling on web text

Before any model is involved, the share of validation tokens where the hint is right grows faster with each doubling: each doubling adds 0.07% of tokens at 1.56M and 0.71% at 100M, where the hint is right on 2.06% of tokens. This matches the blog post's report for the hints, unlike our Shakespeare result. We did not check the cause; near-duplicate pages in both shards are one possibility.

5.2 Leaving out the whole page removes the harm on small corpora

Validation loss without hints was 7.871. Removing only a token's own occurrence does harm below 25M tokens: the overall loss is 7.877 at 0.78M. Removing every match from the token's own page removes this harm. At 100M the loss is 7.816 with the page left out and no hints dropped, against 7.821 with one occurrence left out and half of the hints dropped, and with the page left out dropping half no longer helps (7.825).

5.3 Short matches give most of the gain, and that gain shrinks with training

Adding matches of 2, 3, 4 and 5 tokens raises the share of tokens with a hint from 5% to 92% and gives by far the largest drop, from 7.816 to 7.244 at 100M. Most of this comes from lengths 2 and 3: length 2 alone gives 7.373, and length 4 and longer gives 7.696.

Table 4 shows longer training at corpus 100M. The gain from short matches falls from 0.63 to 0.34 over 16 times more training, because the model learns common next tokens by itself. The gain from lengths 6, 10 and 20 stays between 0.04 and 0.07. The model with all seven lengths depends on them: with hints off it scores 7.754 against 6.916 at the longest run.

Steps	Tokens trained	No hints	Lengths 6, 10, 20	All seven lengths
90	46K	7.871	7.816	7.244
360	184K	7.398	7.333	6.879
1440	737K	6.916	6.872	6.580

Table 4: Validation loss at corpus 100M. Hints leave out the token's own page. The vocabulary grows with the steps (28,359, 39,394 and 46,243 tokens), so compare within a row.

6 Facts and Reasoning

Engram [7] reports gains on reasoning benchmarks as well as knowledge benchmarks, with learned tables, and RETRO's gains largely come from tokens shared with the test data [22]. We found no test of retrieved next-token hints by task kind, so we built a language where the kind of each answer is known.

Setup. The corpus holds 1.5M lines of made-up text: rare facts stated 3 times each, common facts stated 300 times, sums modulo 10 with 20 of 100 pairs held out, two-step comparisons with fresh names, and reading an answer from the line. Hints use lengths 2, 3, 4 and 6, leave out each token's own occurrence, and enter after the embedding, after layer 2 and before the output head. The model has width 64 and 4 layers, and trains for 5760 steps of 32 lines at learning rate 0.02. We test 1000 new lines per group. Chance is 1% for facts, about 10% for sums and 50% for reading and comparison, where always picking the first name scores that much.

Hints	Common	Rare	Other words	Not in table	Held-out sum	Read	Comparison
None	.995	.009	.005	.013	.492	.480	1.000
All 3 places	1.000	1.000	.010	.004	0	.502	.495
All 3 places, half dropped	1.000	1.000	.006	.012	0	.513	.495
Input only	1.000	1.000	.006	.012	0	.513	.497
Length 6 only, all 3 places	1.000	1.000	.012	.010	.730	.474	.499
Output only	1.000	1.000	.010	.003	0	.924	1.000
Output only, half dropped	**1.000**	**1.000**	**.006**	**.012**	**.536**	**.499**	**.999**

Table 5: Accuracy at the answer token.

Results. Table 5 shows that hints take rare facts from chance to 100% when the test uses the stored words, and give nothing when the words change or the fact is absent. Hints at all three places stop the model from learning comparison (.495 against 1.000) and held-out sums (0 against .492). Hint dropout alone does not fix this. Hints only at the output restore comparison, and with half dropped they also restore sums and keep all recall. Section 8 tests why.

This does not contradict Engram: its tables are learned, while our hints are retrieved next tokens. A made-up language shows a mechanism, not its size in real models.

6.1 Web text and reasoning in one model

Next we trained one model on both languages at once, with 2 web chunks and 32 made-up lines per step for 5760 steps. The two languages share no tokens, so one joined hint table gives the same hints as two tables. Web hints use all seven lengths and leave out the token's page, with corpus 100M.

Table 6 shows a trade-off. Hints at all three places give the largest web text gain, 0.28 to 0.31, and block comparison at all three sizes; with hints off their web loss is 0.65 to 0.71 worse than no hints. Output-only hints keep comparison and keep 41% of that gain at width 64 and 64% at width 96. Dropping half restores held-out sums too but keeps only 12%, 7% and 33% of the gain at widths 32, 64 and 96.

7 How Rare Text Must Be for Hints to Help

Recall in language models tracks how often a fact appears [15, 16], and one study reports verbatim recall near 64 exposures at 1B parameters and 16 at 8B [17], while another finds the effect disappears on some models [18]. For retrieval, kNN-LM papers disagree on whether rare patterns [19] or high-frequency tokens [20] gain most. We found no such split for retrieved next-token hints.

Width	Hints	Web, hints on	Web, hints off	Held-out sum	Comparison
32	None	–	7.132	.493	.600
32	All 3 places	6.824	7.846	0	.496
32	Output only, half dropped	7.095	7.107	.458	.929
64	None	–	6.991	.350	.971
64	All 3 places	6.714	7.637	.048	.496
64	Output only	6.878	7.061	0	1.000
64	Output only, half dropped	6.971	6.991	.267	.995
96	None	–	6.956	.484	.981
96	All 3 places	6.677	7.602	0	.495
96	Output only	6.777	6.988	0	.995
96	Output only, half dropped	6.863	6.942	.434	.996

Table 6: One model trained on web text and the made-up language. Accuracy at the answer token. Rare facts in the table scored 1.000 in every hint row.

7.1 A model needs only a few statements of a fact

Each of 200 facts per group was stated exactly 1, 2, 4 up to 256 times in training, with nothing else in the data. The table holds every fact, so a hint is always right. Table 7 shows accuracy without hints. The model reaches 50% at about 45, 19, 10, 5, 4, 3, 3 and 2 statements at widths 16, 24, 32, 48, 64, 96, 128 and 192, so a wider model needs fewer statements. This agrees in direction with [17] and with a width bound for memorizing facts [26]. Going from 2 to 8 layers did not move it, so the count does not follow the number of weights. With 4 times more training the same count gives the same accuracy, so the count matters and not the length of the run. With hints every group scores 100%, but a model trained with hints scores at most 2.5% with hints off, even at 256 statements: it memorized almost nothing itself.

Statements in training	1	2	4	8	16	32
Width 16	.000	.035	.005	.065	.105	.260
Width 24	.020	.025	.075	.175	.395	.885
Width 32	.030	.045	.130	.380	.820	.995
Width 48	.055	.130	.350	.775	1.000	1.000
Width 64	.060	.165	.535	.870	.990	1.000
Width 96	.120	.275	.670	.935	1.000	1.000
Width 128	.125	.310	.735	.970	1.000	1.000
Width 192	.185	.435	.775	.965	.995	1.000
Width 64, 2 layers	.070	.175	.470	.850	.995	1.000
Width 64, 8 layers	.075	.175	.530	.865	1.000	1.000
Width 64, 400 facts per count	.068	.145	.360	.770	.965	1.000
Width 64, 4 times more training	–	–	.485	.885	.995	1.000

Table 7: Accuracy without hints by how many times a fact was stated. Chance is 1%.

7.2 Keeping a model memorizing while it uses hints

A model that always gets hints stops memorizing, so we tried ways to keep both, at width 64 (Table 8). Taking the hints away from a random share of tokens works only at a high share: 25% still leaves 70% accuracy at 256 statements, and 90% matches the model that never saw hints.

Two methods keep 100% with hints and full memorizing. Training every step on the same batch with and without hints matches the model that never saw hints at every count, for 28% more clock time. Keeping only text seen under 16 times in the table gives 100% without hints at 16 statements and above, where the model gets no hint. The closest work we found drops privileged inputs in reinforcement learning [27], and R-Drop also runs two passes per batch, on the same input [28].

Statements in training	1	4	8	16	32	64	256
No hints in training	.060	.535	.870	.990	1.000	1.000	1.000
Hints always	.000	.000	.000	.000	.000	.005	.005
25% of tokens without hints	.005	.010	.015	.005	.010	.140	.700
50% of tokens without hints	.020	.010	.030	.060	.285	.915	1.000
75% of tokens without hints	.045	.235	.555	.960	1.000	1.000	1.000
90% of tokens without hints	.070	.425	.785	.985	1.000	1.000	1.000
No hints in last half	.045	.225	.515	.865	.995	1.000	1.000
No hints in last quarter	.005	.040	.055	.175	.375	.765	1.000
Hints at output only	.020	.000	.010	.000	.010	.030	.010
Every step with and without hints	.065	.475	.890	.985	1.000	1.000	1.000
Table keeps text seen under 16 times	.015	.025	.000	1.000	1.000	1.000	1.000

Table 8: Accuracy with hints off, width 64, one run each. With hints on, every row scores at least 99% except no hints in the last quarter.

7.3 Only training with and without hints survives web text and reasoning

We repeated the best methods in the joint model at width 64 (Table 9). Training every step with and without hints keeps 96% of the web text gain with hints on, and with hints off it beats the model that never saw hints by 0.039 and recovers sums and comparison, for 47% more clock time. With hints on, reasoning stays blocked, and a common fact reaches only 7% with hints off against 78% without hints. A table of only rare text loses the web text gain and comparison stays at chance.

Training	Web on	Web off	Sum on/off	Comparison on/off	Fact off	Gain
No hints	–	6.991	.350	.971	.780	–
All three places	6.714	7.637	.048/.000	.496/.014	.012	100%
With and without hints	**6.726**	**6.952**	.040/.403	.500/.960	.070	96%
Output only, with and without	6.924	6.980	.270/.270	.979/.979	.036	24%
75% of tokens without hints	6.868	6.991	.222/.222	.914/.774	.270	44%
Table under 16 times	7.010	7.077	.581/.581	.495/.505	1.000	0%

Table 9: One model on web text and the made-up language, width 64, 5760 steps, one run each. Hints enter at all three places unless named. Gain is the share of the 0.277 web text gain of hints at all three places. Fact off is a common fact with hints off.

7.4 On web text the gain sits on text the model itself has not seen

We split the gain of 0.357 at 1440 steps over 65K validation tokens (Table 10). Gain per token rises with match length, from 0.22 at length 2 to 3.98 at length 20, but lengths 2 to 4 hold 72% of the gain because they cover far more tokens. Hints that are right, on 18% of tokens, give 125% of the gain, and wrong hints take back 20%.

The model trained on only 737K tokens, 1/135 of the table, so text seen 16 times in the table is usually text the model never saw. Table 11 counts what the model itself saw: 79% of the gain sits on contexts it never saw, and gain per token falls from 0.42 at zero sightings to about zero at 64 or more, which agrees with the facts test. These models sit at a loss near 6.9, so this describes early training only.

Group	Share of tokens	Share of gain	Gain per token
No match	8%	−4%	−0.17
Longest match 2 to 4	80%	72%	0.32
Longest match 5	7%	11%	0.56
Longest match 6	4%	12%	1.05
Longest match 10	0.6%	5%	3.16
Longest match 20	0.3%	3%	3.98
Match seen once	19%	28%	0.53
Match seen 2 to 15 times	35%	39%	0.40
Match seen 16 or more times	38%	37%	0.35

Table 10: Web text gain of hints of all seven lengths. The two blocks split the same tokens.

Times the model saw the context	Share of tokens	Share of gain	Gain per token
0	67.2%	79.2%	0.421
1	7.9%	8.3%	0.377
2 to 3	4.7%	5.1%	0.393
4 to 15	5.0%	3.7%	0.265
16 to 63	2.3%	0.9%	0.138
64 or more	1.5%	−0.1%	−0.026

Table 11: Web text gain split by sightings in the model's own 737K training tokens, for the 88.6% of tokens whose longest match fits inside their chunk.

8 Why Hints Block Reasoning

8.1 The damage comes from hints read before the answer

To find the cause we built a test that runs in about 5 seconds: width 32, 2 layers and 1000 steps on half facts and half comparison lines "a > b . b > c . top ? a .", with the two statements in random order and name triples held out of training. Each model trains every step with and without hints, and we set hints on or off by hand at chosen words of comparison lines, in training and testing (Table 12).

A hint at the answer word does no harm: with hints only there, comparison is .996, and with hints everywhere except there it is .516. The damage comes from hints while the model reads the line. The largest single cause in this test is the dot between the two statements, where the hint says "top", as if the question were starting, and is right 0% of the time. Turning off only that hint gives 1.000, and .991, 1.000 and .974 at width 64, 4 layers and 4000 steps, but .553 at width 128, so it is not the only cause at every size. We did not establish why this hint causes the block. With hints always on and no second pass the model says the first name in the line 94% of the time, which is right half the time. Keeping only hints with one sure next token does not help (.521).

Hints on comparison lines	Comparison, hints on
On everywhere	.764
On only at the answer word	.996
Off only at the answer word	.516
Off at “>”	.671
Off at names	.798
Off at dots	.968
Off at the dot between the statements	1.000
Off on the whole line	1.000

Table 12: Width 32, 1000 steps, one run each. Facts stay near .80 in every row.

8.2 A switch learns to shut wrong hints only when taught

We multiply each token's hint by a switch between 0 and 1, read by a linear layer from the hidden state of a pass without hints. Learned from the loss alone, the switch stays at 1.0 on every word and comparison stays at .769. We then gave it a second target: predict whether the longest hint equals the next token. This target is known from the data and adds one small loss. The switch then drops to .03 at the harmful dot and stays near 1.0 on right hints. Table 13 shows the result by size. Results vary between settings, and hints at the output alone do best in this test. A switch that is fully on or off fixes comparison but drops facts from .80 to .56, because it also shuts right hints. A switch that reads the hidden state where the hint enters, with no second pass, did not beat plain hints at width 128 or at 4000 steps.

Setting	Switch, input	Switch, all three	With and without hints	Hints at output
Width 16	.546	.503	–	–
Width 32	.994	.636	.764	.995
Width 64	.996	.964	.547	1.000
Width 128	.959	.955	.639	1.000
4 layers	.913	.664	.512	.999
4000 steps	.982	.933	.663	1.000

Table 13: Comparison with hints on, one run each. Facts stay near .80 in every run.

8.3 The taught switch also lowers web text loss

We added the taught switch to the joint model, at all three places, training every step with and without hints (Table 14). Comparison with hints on rises from chance to .905, 1.000 and 1.000. Web text loss with hints falls below plain hints by 0.043, 0.070 and 0.094 at widths 32, 64 and 96, so the switch does not trade the web text gain for reasoning. At width 64 this is 125% of the plain gain, and training with and without hints alone reaches 96%, so the extra gain comes from the switch. A common fact with hints off reaches .621 and .957 at widths 64 and 96 against .012 and .011 with plain hints. The switch averages .24 to .27 on web text hints. Held-out sums with hints on stay blocked (0, .251, .094). With the switch at the input only, width 64 gives 6.671 on web text but comparison stays at .495, the reverse of the small test.

The lead depends on training length. At width 64 the switch is behind at 360 steps (7.155 against 6.921), level at 1440 (6.819 against 6.816) and ahead at 5760 (6.644 against 6.714), and comparison is not yet learned below 5760 steps. At 1440 steps a switch with no training pass without hints

gives no gain and comparison .007; we did not run that arm at 5760 steps. The widths span only 3.2 times the parameters, each setting ran once, and the switch needs a pass without hints at every step, so it costs clock time we did not measure under equal load.

The closest work we found has no such target. Engram's gate reads the hidden state and the memory [7] and we found no separate target for it in the paper. kNN-Adapter learns a per-token mix on top of a frozen pretrained model [24], and SpecDec++ trains an acceptance head for speculative decoding at inference [25]. We found no gate trained during pretraining on whether the retrieved token is right.

Width	Web, no hints	Web, plain hints	Web, switch	Web, switch off	Comparison, plain/switch	Fact off
32	7.132	6.824	6.781	7.082	.496/.905	.014
64	6.991	6.714	**6.644**	6.952	.496/1.000	.621
96	6.956	6.677	**6.582**	6.918	.495/1.000	.957

Table 14: One model on web text and the made-up language, 5760 steps, one run each. Fact off is a common fact with hints off in the switch model.

9 Limits

- The web text models stop at a loss far above real models, so they do not show what hints do for a trained model. The largest joint model has 4.3M parameters.
- Each setting was run once, and learning rates were picked on the same validation chunks used for reporting.
- Shakespeare is not web text: it has no crawled duplicates and its tokens are characters, so our result there does not contradict the blog post's rising returns.
- In a real run, leave-one-out needs the full corpus before training starts, or a table large enough to subtract one count per lookup.
- We measured loss at equal steps, not clock time, except where stated.

10 Open Questions

1. If a GPT-2-sized model is trained with Muon on web text, does adding n-gram hints from the training data lower validation loss compared with the same model without hints?
2. If a model is trained on web text with a hint corpus about 500 times larger than the tokens it trains on, does leaving out the token's page and dropping half of the hints beat hints from earlier steps?
3. If a model is trained on web text with hints, does each doubling of the hint corpus lower validation loss by more than the doubling before it, after near-duplicate documents are removed?
4. If a model is trained with hints, does it repeat training text word for word more often than the same model without hints?
5. If a model trains far longer than our web text runs, does the validation loss gain from short matches disappear?

6. If a hint table drops n-grams that the model itself has already seen often, how much of the web text gain is kept?

7. If a GPT-2-sized model is trained on web text with a switch taught to predict whether each hint is right, is its validation loss lower than with plain hints, and does the lead keep growing with training length?

8. If the switch is trained with no pass without hints, does it still lower web text loss at long training?

11 Conclusion

For hints from a full corpus, remove each token's own occurrence (on web text, its whole page) and drop half of the hints during training. In a small Muon model this beat hints from earlier steps by about 0.08 at three sizes, and by more when the corpus was far larger than the tokens trained. If a run makes more than one pass, causal hints need the same removal. If hints must not replace the model's own learning, train every step with and without hints, and add a switch trained to predict whether each hint is right. In our joint test this restored comparison with hints on and lowered web text loss further at the longest training, though sums stayed blocked with hints on. Hints at the output only need no second pass and keep reasoning, at the cost of part of the gain. These are small models and single runs, so all of this must be tested in real training on FineWeb.

References

[1] H. Brunborg. N-gram hinted language modeling: NanoGPT Speedrun in 6.7s. `https://brunborg.com/n-gram-hinted-lm`, 2026.

[2] Track 1: Retrieval-augmented n-gram hints. Pull request 367, `https://github.com/KellerJordan/modded-nanogpt/pull/367`.

[3] H. Brunborg. n-gram-hinted-lm, file `old_records/online_hints.py`. `https://github.com/hermabr/n-gram-hinted-lm`.

[4] K. Jordan. Muon: an optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024.

[5] Muon is scalable for LLM training. arXiv:2502.16982, 2025.

[6] Wang et al. Muon outperforms Adam in tail-end associative memory learning. arXiv:2509.26030, 2025.

[7] Conditional memory via scalable lookup: a new axis of sparsity for large language models. arXiv:2601.07372, 2026.

[8] Tensorizing Engram: sharing latents across n-gram embeddings is beneficial in LLMs. arXiv:2606.08347, 2026.

[9] Lngram v2: latent n-gram memory with interpretable discrete representations. arXiv:2609.03426, 2026.

[10] Improving language models by retrieving from trillions of tokens. arXiv:2112.04426, 2021.

[11] D. Micci-Barreca. A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. SIGKDD Explorations, 2001.

[12] L. Prokhorenkova et al. CatBoost: unbiased boosting with categorical features. NeurIPS, 2018.

[13] N. Srivastava et al. Dropout: a simple way to prevent neural networks from overfitting. JMLR, 2014.

[14] V. Rosić. Muon lab. `https://github.com/vukrosic/muon-lab`, 2026.

[15] N. Kandpal et al. Large language models struggle to learn long-tail knowledge. ICML, 2023.

[16] Tracing multilingual factual knowledge acquisition in pretraining. arXiv:2505.14824, 2025.

[17] Positional fragility in LLMs: how offset effects reshape our understanding of memorization risks. arXiv:2505.13171, 2025.

[18] Memory vs. context? Influential factors of factual recall in language models. arXiv:2609.24238, 2026.

[19] Generalization through memorization: nearest neighbor language models. arXiv:1911.00172, 2019.

[20] Long-tail crisis in nearest neighbor language models. arXiv:2503.22426, 2025.

[21] X. Wang et al. Generalization v.s. memorization: tracing language models' capabilities back to pretraining data. ICLR, 2025. arXiv:2407.14985.

[22] T. Norlund et al. On the generalization ability of retrieval-enhanced transformers. arXiv:2302.12128, 2023.

[23] V. Rosić. ngram-hints-llm: code for this paper. `https://github.com/vukrosic/ngram-hints-llm`, 2026.

[24] kNN-Adapter: efficient domain adaptation for black-box language models. arXiv:2302.10879, 2023.

[25] SpecDec++: boosting speculative decoding via adaptive candidate lengths. arXiv:2405.19715, 2024.

[26] S. Ravfogel, G. Yehudai et al. Geometric Factual Recall in Transformers. arXiv:2605.12426.

[27] Privileged Information Dropout in Reinforcement Learning. arXiv:2005.09220.

[28] X. Liang, L. Wu et al. R-Drop: Regularized Dropout for Neural Networks. NeurIPS 2021.